CkaDinTi
Edin's Project Tasks

Owner: student web, LAN game host, realtime gameplay, authoritative scoring, and runtime analytics

Build the authoritative classroom runtime that lets browser students join and play safely over the teacher's LAN.

Project foundation

The product is a desktop-first, local-first AI classroom quiz platform. The supplied deep-research PDF is the product brief; this document is the actionable task assignment for this team member.

Shared technology: Electron + React + TypeScript for the teacher app; React + Vite for students; SQLite + Drizzle; Fastify + Socket.IO; Zod contracts; OpenAI structured outputs with mandatory teacher review.

Milestone A - runtime foundation

- Scaffold the student React/Vite app and embedded Fastify + Socket.IO host package.
- Implement join-code creation, QR/LAN URL payload, lobby membership, host start/pause/resume/end, and reconnect handling.
- Use explicit acknowledgements, idempotency keys, and server-side deadlines for answer submissions.
- Define Zod contracts for socket events and HTTP health/network-readiness checks.

Milestone B - authoritative game

- Create immutable approved-question snapshots for each game session.
- Implement Cyber Raid phases and authoritative scoring: base 1000, speed bonus 0-60, capped streak bonus 0-40, and no ranking score from power-ups.
- Ensure students receive no correct answer or authoritative score before reveal.
- Persist player answers, question results, session events, and aggregate analytics.

Milestone C - student and host UX

- Build student Join, Lobby, Game, Feedback/Explanation, Mission Progress, and Results screens.

- Build teacher live controls: connected players, lobby lock, kick, round state, answer count, pause/resume, emergency end, and private analytics drawer.
- Add projector-safe output that shows only intended public information.
- Add runtime integration tests and Playwright teacher/student workflows.

Acceptance criteria

- Multiple browser clients can join a LAN game, answer the same authoritative round, reconnect, and receive the same reveal.
- Duplicate or late submissions cannot corrupt score or results.
- Historical results remain stable after a quiz is edited.
- Tests cover timing, reconnects, scoring invariants, privacy boundaries, and session analytics.

Non-overlap rules

- Do not implement teacher review, AI generation, lesson parsing, or teacher-side content repositories; Arron owns those areas.
- Own game/session/result/analytics table families. Coordinate any migration that touches Arron's lesson/quiz tables.
- Consume approved quiz snapshots through packages/contracts; do not reimplement teacher review logic.
- If using Graphify, keep it behind a mission/report adapter and never use it for authoritative game state or scoring.

Workflow

- Work only on the matching branch: Edin.
- Use RTK wrappers for repository inspection and test output: rtk git, rtk rg, rtk read, and rtk test.
- Keep commits focused and prefix feature commits with the owner name.
- Update docs/IMPLEMENTATION_STATUS.md when a milestone is completed.
- Never commit API keys, student personal data, or generated SQLite files.

References: Graphify (optional visualization adapter) - https://github.com/Graphify-Labs/graphify
RTK (token-efficient CLI workflow) - https://github.com/rtk-ai/rtk
Repository - https://github.com/CkaDinTi/CkaDinTi